Cryptography
Practical 5: Stream Ciphers

Martin Harrigan, ✉ martin.harrigan@setu.ie

In this practical we will use Python's `pyca/cryptography` package to experiment with stream ciphers. You should familiarise yourself with the API documentation at ↗ `https://cryptography.io`. In particular, the documentation for stream ciphers is available under The Hazardous Materials Layer ▸ Primitives ▸ Symmetric Encryption.

1 RC4

Section A provides sample Python code for encrypting and decrypting using RC4. Note that RC4 requires only a key (no nonce). The key can be 40, 56, 64, 80, 128, 160, 192, or 256 bit. RC4 is *not* considered secure and *should not* be used in real-world applications.

Note that the `rc4_encryptor` and `rc4_decryptor`, and the corresponding objects for ChaCha20 (see Section 2) and the Advanced Encryption Standard (AES) in Counter Mode (see Section 3), return all of their output from `update()`. The code still calls `finalize()`, as the API requires, but it returns no further bytes since there is no concept of a block. Additionally, there is no need for padding or ciphertext stealing.

The following ciphertext in hexadecimal notation was produced using the RC4 algorithm with the key `thisisonelongkey`:

```
eb 3e c6 68 de 7d 9e 02
38 55 44 b9 69 1b c3 8b
9d 56 ab 7d 9d 53 e8 02
c1 86 85 1a d0 5e ad d0
97 2a 75 56 e1 96 2a 60
a1 f6 b2 86 bc 81 11 4a
ac bd
```

What is the corresponding plaintext?

2 ChaCha20

Section B provides sample Python code for encrypting and decrypting using ChaCha20. It requires a 256 bit key and a 128 bit value that `pyca/cryptography` calls the nonce: a 32 bit block counter followed by a 96 bit nonce, as in RFC 7539. ChaCha20 is a counter-based stream cipher, and the counter tells it which block of the keystream to produce. It is considered secure and can be used in real-world applications.

The following ciphertext in hexadecimal notation was produced using the ChaCha20 algorithm with the key `averylongkeymakesforaverygoodkey` and the nonce `8ec1f037a3f7322e65cf3319a73d0170`:

```
e8 e7 1b 9c 51 82 d5 07
c6 d1 0f f0 6c 9b 72 07
74 d3 f9 4f f2 32 d3 48
44 f6 e5 94 cf 27 fa ee
36 a8 d1 1d 9d ea cd b4
1d 55 81 ea 1e 7d 1c e2
3b fe fc d0 c9 96 3e aa
e4 12 04 de ee 92 1c d5
b8 c9 21 25 bd 71
```

What is the corresponding plaintext? Could you decrypt this ciphertext without knowing the nonce?

3 AES in Counter Mode

Section C provides sample Python code for encrypting and decrypting using AES in Counter Mode. It requires a 128, 192, or 256 bit key and a 128 bit initial counter block. The code fills it with a 96 bit nonce followed by a 32 bit counter that starts at zero, and the counter increases by one for each block. It is considered secure and can be used in real-world applications.

Counter mode transforms a block cipher, such as AES, into a stream cipher. The choice between a block cipher in Counter Mode, such as AES, and a "dedicated" stream cipher, such as ChaCha20, depends on several factors such as availability of implementations, hardware support, and performance requirements.

Compare the performance of the implementations of ChaCha20 and AES in Counter Mode in the above examples. Which is faster? You can generate suitably long plaintext messages using:

```
import os
...
plaintext_bytes = os.urandom(10000000)
```

Additionally, you can time your code using:

```
import time
...
before = time.perf_counter()
# code you wish to time
after = time.perf_counter()
print(f"{after - before:0.4f} seconds")
```

4 Reusing a Key and Nonce

A stream cipher turns a key and a nonce into a keystream, so the same key and nonce always give the same keystream. This is why the lecture warns you never to reuse a key-nonce pair. Modify the code in Section B so that it encrypts two plaintexts of the same length under the same key and the same nonce. XOR the two ciphertexts together and compare the result with the XOR of the two plaintexts. What do you notice? How does this relate to the Many-Time Pad in Practical 3?

5 Integrity

Block ciphers (e.g., AES) and stream ciphers (e.g., ChaCha20) provide confidentiality but they do not provide any inherent integrity protection. In other words, it may be possible for Eve to modify a ciphertext that was sent from Alice to Bob and for that modification to go undetected by Bob. For a block cipher in a mode such as CBC, a modification of a single bit will garble the decryption of an entire block (and flip the same bit in the next block). This may be detected at the application level. However, for a stream cipher, including a block cipher in Counter Mode, a modification of a single bit will flip a single bit of the decryption. This may not be detected at the application level. We will revisit this problem in a later lecture.

Modify a single bit in each of the above ciphertexts and examine the impact on the resulting plaintext.

5.1 Tampering with a Ciphertext

Flipping a bit at random changes the plaintext in a way that Eve does not control, but she can do better if she can guess the plaintext. Alice's bank accepts payment instructions in a fixed format, such as `Pay Bob EUR 0100`: the payee's name in three letters and the amount in four digits. Eve knows that Alice owes Bob 100 EUR. When Alice sends her bank the following ciphertext, encrypted using ChaCha20 under a key and nonce that Eve does not know, Eve guesses that the plaintext is exactly `Pay Bob EUR 0100`:

```
c2 61 53 2e 6b 9f 23 79
17 c6 ab ce f1 c4 54 e9
```

Without the key or the nonce, change the ciphertext so that it decrypts to `Pay Eve EUR 9900`. Why does the attack need neither? To check your answer, decrypt it with the key and nonce below, which Eve would never see:

```
key = bytes("onlyaliceandbobshouldholdthiskey", "utf-8")
nonce = bytes.fromhex("0000000000000004d2f8a1c93e05b7d")
```

Appendices

A Python Code for Encrypting and Decrypting Using RC4

```
from cryptography.hazmat.decrepit.ciphers.algorithms import ARC4
from cryptography.hazmat.primitives.ciphers import Cipher

# RC4 (ARC4) is a stream cipher, now considered broken due to biases in
# its keystream; prohibited in TLS since RFC 7465.

key = "mykey"
key_bytes = bytes(key, "utf-8")
print("Key: " + key)

rc4_cipher = Cipher(ARC4(key_bytes), mode=None)
rc4_encryptor = rc4_cipher.encryptor()
rc4_decryptor = rc4_cipher.decryptor()

plaintext = "someplaintext"
plaintext_bytes = bytes(plaintext, "utf-8")
print("Plaintext: " + plaintext)

ciphertext_bytes = rc4_encryptor.update(plaintext_bytes) + rc4_encryptor.finalize()
ciphertext = ciphertext_bytes.hex()
print("Ciphertext: " + ciphertext)

plaintext_bytes_2 = rc4_decryptor.update(ciphertext_bytes) + rc4_decryptor.finalize()
plaintext_2 = str(plaintext_bytes_2, "utf-8")
print("Original Plaintext: " + plaintext_2)
```

Listing 1: rc4_encrypt_decrypt.py

B Python Code for Encrypting and Decrypting Using ChaCha20

```
import os

from cryptography.hazmat.primitives.ciphers import Cipher, algorithms

# ChaCha20 is a stream cipher (no padding needed).
# The raw API takes 16 bytes: a 4-byte little-endian block counter
# followed by a 12-byte nonce, as in RFC 7539.

key_bytes = os.urandom(32) # Fresh key for each standalone demonstration.

nonce_bytes = bytes(4) + os.urandom(12)

chacha20_cipher = Cipher(algorithms.ChaCha20(key_bytes, nonce_bytes), mode=None)
chacha20_encryptor = chacha20_cipher.encryptor()
chacha20_decryptor = chacha20_cipher.decryptor()

plaintext = "someplaintext"
plaintext_bytes = bytes(plaintext, "utf-8")
print("Plaintext: " + plaintext)

ciphertext_bytes = chacha20_encryptor.update(plaintext_bytes) + chacha20_encryptor.finalize()
ciphertext = ciphertext_bytes.hex()
print("Ciphertext: " + ciphertext)

plaintext_bytes_2 = chacha20_decryptor.update(ciphertext_bytes) + chacha20_decryptor.finalize()
plaintext_2 = str(plaintext_bytes_2, "utf-8")
print("Original Plaintext: " + plaintext_2)
```

Listing 2: chacha20_encrypt_decrypt.py

C Python Code for Encrypting and Decrypting Using AES in Counter Mode

```
import os

from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

# AES in CTR mode turns a block cipher into a stream cipher by
# encrypting successive counter values and XORing with plaintext.
# No padding needed since it operates as a stream cipher.

# Fresh key for this standalone run; never overlap counter ranges under a key.
key_bytes = os.urandom(32)
# 96-bit message nonce followed by a 32-bit initial counter, big-endian.
# Limit messages to fewer than 2**32 blocks and do not reuse message nonces.
nonce_bytes = os.urandom(12) + bytes(4)

aes_ctr_cipher = Cipher(algorithms.AES(key_bytes), mode=modes.CTR(nonce_bytes))
aes_ctr_encryptor = aes_ctr_cipher.encryptor()
aes_ctr_decryptor = aes_ctr_cipher.decryptor()

plaintext = "someplaintext"
plaintext_bytes = bytes(plaintext, "utf-8")
print("Plaintext: " + plaintext)

ciphertext_bytes = aes_ctr_encryptor.update(plaintext_bytes) + aes_ctr_encryptor.finalize()
ciphertext = ciphertext_bytes.hex()
print("Ciphertext: " + ciphertext)

plaintext_bytes_2 = aes_ctr_decryptor.update(ciphertext_bytes) + aes_ctr_decryptor.finalize()
plaintext_2 = str(plaintext_bytes_2, "utf-8")
print("Original Plaintext: " + plaintext_2)
```

Listing 3: aes_ctr_encrypt_decrypt.py